

Music Bot Documentation

Discord multi-worker music bot: `main.py` controller, `worker.py` workers, `db.py` storage and `appsettings.py` settings.

This document explains every function, class and method of the source files, the settings you can change, the rules for who can control playback, the saved-songs ("algo") and autoplay system, and the messages the controller and the workers exchange.

Contents

1. [Architecture overview](#)
2. [Configuration](#)
3. [Commands](#)
4. [Who can control playback](#)
5. [Saved songs \(algo\) and autoplay](#)
6. [main.py reference \(controller\)](#)
7. [worker.py reference \(worker\)](#)
8. [db.py and appsettings.py reference](#)
9. [IPC operations and state events](#)
10. [Tunable constants and files](#)
11. [Deployment and service](#)

1. Architecture overview

The system has four parts that run together on the server:

Part	File / service	Job
Controller	<code>main.py</code> (systemd unit <code>SERVER_musicbots</code>)	Logs in as the main bot, registers the slash commands for every server, enforces the whitelist, reads the dev <code>prefix</code> commands, starts and watches the workers, decides which worker serves which request, and relays commands to them.
Workers	<code>worker.py</code> , started once per worker token with <code>--index N</code>	Each copy is a separate Discord bot in its own process. Worker N uses <code>tokens[N]</code> . A worker keeps one session per server , so it can play in one voice channel in every server at the same time. It searches, resolves streams, plays, records listens and runs autoplay.
Storage	<code>db.py</code> → <code>database/</code>	SQLite files. <code>database/server/servers.db</code> holds per-server settings; <code>database/algo/<server_id>.db</code> holds every user's saved songs for that server.
Lavalink	<code>localhost:2333</code> (unit <code>SERVER_lavalink</code>)	The audio server, with the youtube-plugin and lavasrc-plugin. Workers connect to it with wavelink. YouTube streams are resolved by yt-dlp first and given to Lavalink as plain URLs.

How a play request flows

1. A user runs `/play` in Discord. The controller checks the server is whitelisted and the control channel (`_guard`).
2. The controller takes the pick lock, asks every worker for its real state (`sync`), and chooses a worker and voice channel (`_pick_for_play`). A new worker is *reserved* for that user before the lock is released, so two simultaneous `/play`s never get the same worker.
3. The controller sends `search_and_play` to that worker over its Unix socket `.worker<N>.sock`, including the user id.
4. The worker creates a session for that server (the user becomes its **controller**), searches (Spotify for clean metadata, YouTube Music for candidates), scores the candidates, connects to the voice channel and plays the best one. YouTube tracks are resolved to a direct stream URL with yt-dlp. The worker sets the voice channel status to "🎵Playing -